Begonnen am	Freitag, 24. Januar 2025, 10:16
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 12:13
Verbrauchte Zeit	1 Stunde 56 Minuten

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

Frage 1

Nicht beantwortet

Nicht bewertet

Parameter Passing

Parameters can be passed through global variables. Consider the following questions:

1. Is passing parameters by global variables an efficient way? Why?
2. Do you consider global variables as a maintainable way for parameter passing? What speaks for or against this method?
3. What are possible risks when using global variables?

1. Passing parameters by global variables is not efficient, because accessing the values has considerable overhead. The address needs to be loaded before the value can be loaded or stored. this also needs more program memory for the literal pool.
2. Global variables in general are hardly maintainable. The developer must avoid name collisions between all used modules, thus requiring unique names.
3. There is no way to control who can access or modify a global variable.

Frage 2

Richtig

Erreichte Punkte 5,00 von 5,00

Parameter Passing

Depending on the parameter data type or the number of parameters, different methods of parameter passing are chosen by the compiler. For the following parameter types, select the best method.

1. Pass four uint8_t values to a subroutine. The subroutine does not need to modify the values:
   uint8_t build_snowman(uint8_t height, uint8_t weight, uint8_t nr_of_noses, uint8_t nr_of_eyes);
   Pass the values in R0-R3 ✓

2. Pass **two uint32_t** variables to a subroutine. The subroutine **needs to edit** the values:
   void change_password(uint32_t *part_a, uint32_t *part_b);
   Use R0-R1 to pass the adresses of the variable ✓

3. Pass **twelve uint8_t** variables (pass by value):
   void cast_ice_ray(uint8_t speed, uint8_t range, uint8_t target, …);
   Use R0-R3 to pass four values, push the rest or ✓

4. Pass the uninitialized **uint32_t anna[4]** array to a subroutine. The subroutine will fill the array:
   Pass the array address via R0. ✓

5. Pass the string **char elsa = "Let it go!"** to a subroutine to change all letters to uppercase:
   Pass the address of elsa in R0. ✓

Richtig

Bewertung für diese Einreichung: 5,00/5,00.

Frage 3

Richtig

Nicht bewertet

Parameter Passing: Register roles

The ARM Procedure Call Standard defines the use of the available Registers in the ARM Cortex-M0 processor. Select the appropriate Registers for the following roles:

Role	Registers
passing arguments	R0-R3
passing the result	R0-R3
local variables	R4-R7

R0-R3 are designated for argument passing and are also used for return values. Depending on the data type of the return value, one ore more registers need to be used.
R4-R11 are designated for local variables.
R12 may be used by the linker as scratch register, but the course does not cover this.
R13 is the Stack Pointer (SP)
R14 is the Link Register (LR)
R15 is the Program Counter (PC)

Register Usage

Register	Synonym	Role
r0	a1	Argument / result / scratch register 1
r1	a2	Argument / result / scratch register 2
r2	a3	Argument / scratch register 3
r3	a4	Argument / scratch register 4
r4	v1	Variable register 1
r5	v2	Variable register 2
r6	v3	Variable register 3
r7	v4	Variable register 4
r8	v5	Variable register 5
r9	v6	Variable register 6
r10	v7	Variable register 7
r11	v8	Variable register 8
r12	IP	Intra-Procedure-call scratch register[1)]
r13	SP	
r14	LR	
r15	PC	

Register contents might be modified by callee

Callee must preserve contents of these registers (Callee saved)

Cortex-M0: Registers r8 – r11 have limited set of instructions. Therefore, they are often not used by compilers.

Frage 4

Richtig

Nicht bewertet

Stack frame

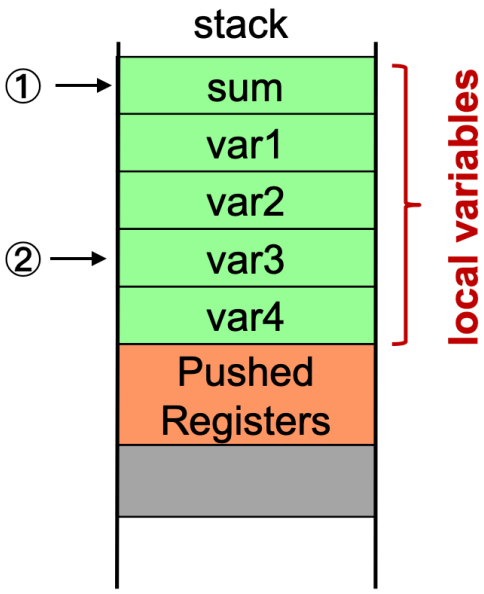
Items on the stack can be accessed with stack pointer relative addressing. Write Assembly code to access the local variables marked in the stack frame shown below.

Store contents of Register R4 in 'sum' (1):

STR R4,[SP,#0x00]

Read 'var3' (2) into Register R7:

LDR R7,[SP,#0x0c]



Do not use PUSH and POP to access local variables, as this will change the SP.

Instead, use Stack Pointer relative addressing: STR Rx, [SP,#<offset>] and LDR Rx,[SP,#<offset>].

Frage 5

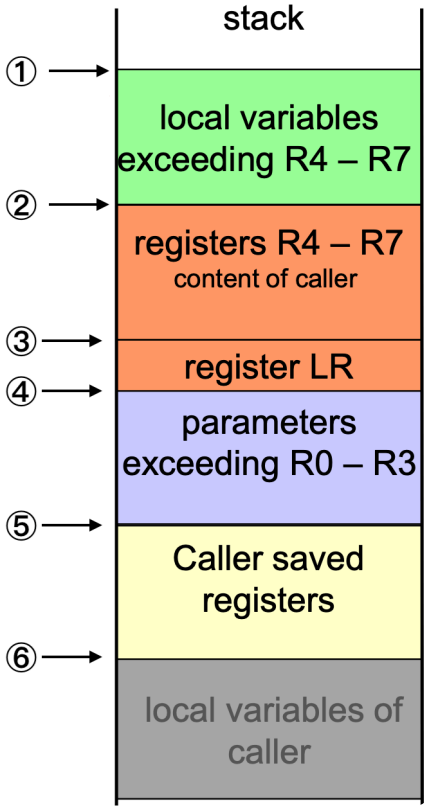
Richtig

Nicht bewertet

Stackframe – Range

Each subroutine has a stack frame. In the following image, the stackframe of the active subroutine ranges from:

1 to 4



The stack frame of a subroutine ranges from the first to the last word on the stack that **the subroutine itself pushed** on the stack.

Frage 6

Richtig

Nicht bewertet

Parameter Passing – Responsibilities of Caller and Callee

The ARM Procedure Call Standard defines the responsibilities of who must save which registers. For the following registers of the ARM Cortex-M0, select who must preserve the stored values to ensure that the calling function can continue its work after the return of the subroutine call.

Subroutine call responsibilites

Registers	Responsible
R0-R3 at function call	saved by Caller
R4-R7 at function call	saved by Callee

R0-R3 at return	restored by Caller
R4-R7 at return	restored by Callee
LR at function call	saved by Callee
LR at return	restored by Callee

For the following registers, select whether the subroutine (Callee) is allowed to return without restoring the contents of the Register to the state when called.

R0

may be modified when returning

R6

must contain same value when returning as when ent

SP

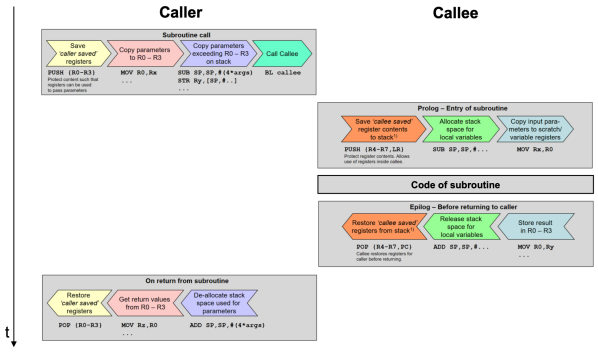
must contain same value when returning as when ent

Registers R0-R3 are defined to be saved by the Caller. They may be changed by the subroutine, as the return value is passed through these registers.

Registers R4-R11 are variable registers. If the subroutine modifies these registers, their contents must be restored before returning.

The SP must be restored to the same value as on entry, because the stack frame of the callee must be torn down.

The LR is used to keep track of the address of the instruction to return to. It must be stored when the callee calls another subroutine. Upon return, the return address is loaded into the PC.



Parameter Passing

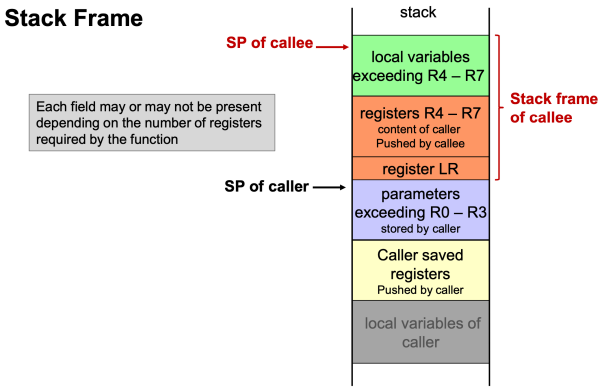
Drag the items in the order that they would appear in on the stack.

- additional local variables of callee
- R4-R7
- LR
- Parameters that don't fit in R0-R3
- R0-R3 of caller

Die Antwort ist richtig.

The stack frame contains the elements of each subroutine that it needs for its execution or in preparation of the next subroutine call. Not all elements shown below will be present in every subroutine call.

- If a subroutine has very few local variables, it may not be necessary to use the stack, and all local variables can be stored in registers.
- If a subroutine does not contain further subroutine calls, it is unnecessary to store the Link Register on the stack.
- If the called subroutine has very few or no parameters, no room on the stack is required for parameters.



◀ Lab Subroutines and Parameter Passing

Direkt zu:

Lecture Modular Coding and Linking ▶